LAB 7 – FUZZ TESTING FOR INPUT VALIDATION BUGS

Name: Pooja L R
SRN: PES1UG24AM475
Branch: CSE (AIML)
Subject: Software Engineering Lab
Lab Number: 7

---

1. OBJECTIVE

To explore fuzz testing as a security and quality assurance technique by identifying and fixing input validation bugs in a Python application using the Hypothesis fuzzing library.

The objective is to:

- Understand the purpose and methodology of fuzz testing.
- Apply property-based fuzzing using Hypothesis.
- Detect and analyse bugs caused by edge-case inputs.
- Improve code robustness through defensive programming.

2. SOFTWARE REQUIREMENTS

- Python 3.13
- Hypothesis
- Pytest
- Windows PowerShell

3. INTRODUCTION

Fuzz testing is a dynamic testing technique in which random, unexpected or malformed inputs are provided to an application to discover bugs, crashes and vulnerabilities.

Hypothesis is a property-based testing library for Python that automatically generates test inputs and identifies edge cases that may cause application failures.

In this experiment, Hypothesis was used to test three functions in the loan eligibility application.

4. FUNCTIONS TESTED

1. parse_income()

This function parses monthly income entered by the user. It was tested with valid income values, malformed inputs, empty strings and non-string inputs.

2. monthly_emi()

This function calculates the monthly EMI based on loan principal, interest rate and loan tenure. It was tested with invalid formats, missing values, invalid numerical values and zero-interest loans.

3. pan_category()

This function identifies the PAN category based on the fourth character of the PAN number. It was tested with valid PAN numbers, invalid formats and unexpected inputs.

5. IMPLEMENTATION AND EXECUTION

Step 1: Installing Dependencies

The required Python libraries were installed using:

python -m pip install hypothesis pytest

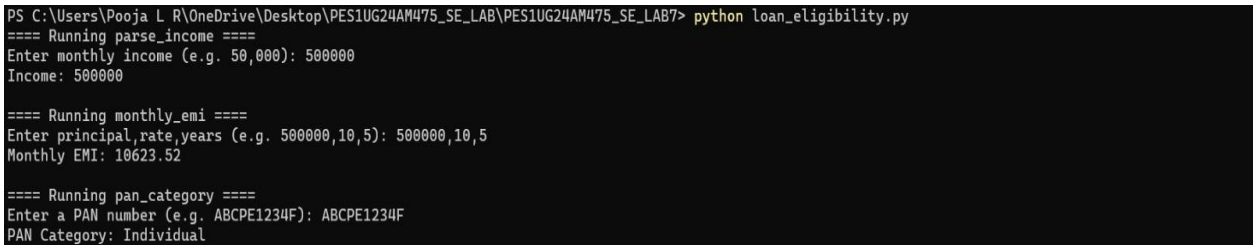
Step 2: Running the Original Application

The original buggy application was executed using:

python loan_eligibility.py

Valid sample inputs were provided to verify the original application.

Screenshot 1 – Original Application Execution



Step 3: Implementing Hypothesis Tests

The test file test_loan_eligibility.py was completed using the Hypothesis library.

The @given decorator and st.text() | st.none() strategy were used to generate unexpected inputs automatically for all three functions.

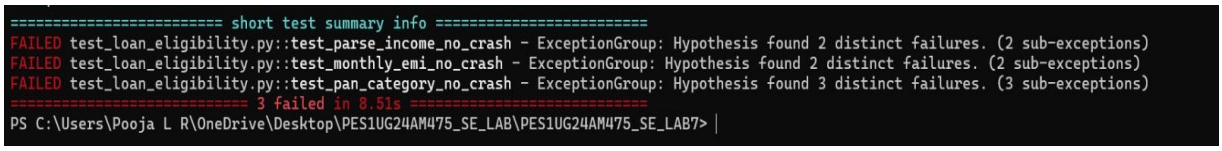
Step 4: Running the Initial Fuzz Tests

The initial tests were executed using:

python -m pytest test_loan_eligibility.py -v

Hypothesis identified failures in all three functions due to improper handling of unexpected inputs.

Screenshot 2 – Initial Failing Test Cases



Step 5: Fixing the Application

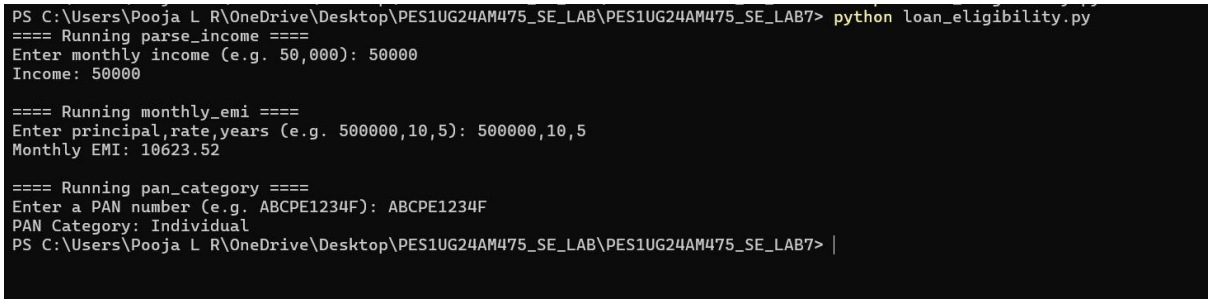
The bugs in loan_eligibility.py were fixed using defensive programming techniques.

The following improvements were implemented:

- Added input type and format validation for monthly income.
- Added exception handling for invalid EMI inputs.
- Handled zero-interest loan calculations separately.
- Added validation for principal, interest rate and loan tenure.
- Added PAN format validation.
- Used safe dictionary lookup for PAN category identification.
- Invalid inputs return None instead of causing application crashes.

The corrected application was executed again using: python loan_eligibility.py

Screenshot 3 – Fixed Application Execution



Step 6: Re-running the Tests

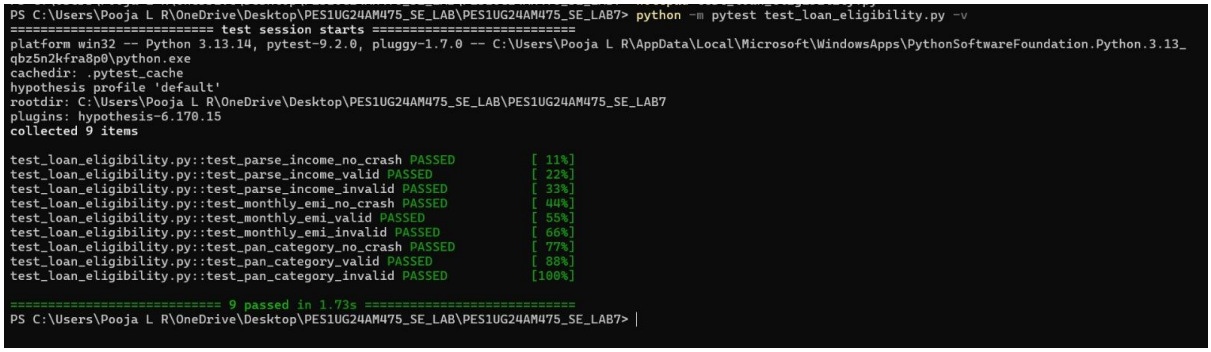
After fixing the application, the test suite was executed again using:

python -m pytest test_loan_eligibility.py -v

Additional tests were included to verify valid and invalid inputs for all three functions.

The final execution completed successfully with all nine tests passing.

Screenshot 4 – Final Passing Test Results



6. TEST RESULTS

The following table summarises the final test results.

Function	Test Cases	Result
parse_income()	3	PASSED
monthly_emi()	3	PASSED
pan_category()	3	PASSED

Total	**9**	**9 PASSED**

Final Test Execution:

- Total tests collected: 9
- Tests passed: 9
- Tests failed: 0
- Execution time: 1.73 seconds

7. OBSERVATIONS

1. The original application worked correctly with valid inputs but failed when unexpected inputs were provided.
2. Hypothesis automatically generated unexpected inputs and exposed failures in all three functions.
3. Input validation and exception handling improved the robustness of the application.
4. Additional valid-input and invalid-input test cases helped verify the corrected implementation.
5. All nine tests passed after fixing the application.

8. CONCLUSION

In this lab, fuzz testing was successfully performed on a Python loan eligibility application using the Hypothesis library.

The experiment demonstrated how property-based testing can identify input validation bugs and edge cases that may not be covered by ordinary testing.

The three functions, parse_income(), monthly_emi() and pan_category(), were tested and improved using defensive programming techniques.

The final test execution resulted in **9 successful test cases**, demonstrating improved application robustness against unexpected and invalid inputs.